

IP COMMAND

DEEP DIVE



Prepared By: Hosni Zaaraoui

OBJECT: LINK

When managing Linux systems, understanding network interfaces is essential — and the `ip link` object is where it all begins.

It's the part of the `ip` command family that controls your network interfaces — physical, virtual, or loopback.

Whether you want to bring an interface up, rename it, or change its MTU, this is your go-to tool.

WHAT IT DOES

The link object lets you view and modify network interfaces on your system.

It replaces the old `ifconfig` functionality and offers much more power, consistency, and scripting flexibility.

You can:

- List available interfaces
- Bring interfaces up or down
- Change parameters like MTU or queue discipline
- Rename interfaces
- Inspect advanced stats or errors

BASIC SYNTAX

```
ip link [ show | set | add | delete ] [dev DEVICE] [options]
```

COMMON COMMANDS EXPLAINED

ip link show

Displays all interfaces and their states (UP/DOWN, MAC address, MTU, etc.)

```
hosni ~ 16:14 ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eno1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN mode DEFAULT group default qlen 1000
   link/ether f0:1f:af:59:ea:5a brd ff:ff:ff:ff:ff:ff
   altname enp0s25
3: wlp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DORMANT group default qlen 1000
   link/ether 34:23:87:06:ee:69 brd ff:ff:ff:ff:ff:ff
4: virbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN mode DEFAULT group default qlen 1000
   link/ether 52:54:00:9c:06:84 brd ff:ff:ff:ff:ff:ff
5: ham0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1404 qdisc fq_codel state UNKNOWN mode DEFAULT group default qlen 1000
   link/ether 7a:79:00:00:00:00 brd ff:ff:ff:ff:ff:ff
6: vmnet1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN mode DEFAULT group default qlen 1000
   link/ether 00:50:56:c0:00:01 brd ff:ff:ff:ff:ff:ff
7: vmnet2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN mode DEFAULT group default qlen 1000
   link/ether 00:50:56:c0:00:02 brd ff:ff:ff:ff:ff:ff
```

ip -s link show

Shows detailed interface statistics (RX/TX packets, errors, drops)

```
hosni ~ 16:14 ip -s link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   RX: bytes packets errors dropped missed mcast
       949276    9849      0      0      0      0
   TX: bytes packets errors dropped carrier collsns
       949276    9849      0      0      0      0
2: eno1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN mode DEFAULT group default qlen 1000
   link/ether f0:1f:af:59:ea:5a brd ff:ff:ff:ff:ff:ff
   RX: bytes packets errors dropped missed mcast
         0         0      0      0      0      0
   TX: bytes packets errors dropped carrier collsns
         0         0      0      0      0      0
   altname enp0s25
3: wlp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DORMANT group default qlen 1000
   link/ether 34:23:87:06:ee:69 brd ff:ff:ff:ff:ff:ff
   RX: bytes packets errors dropped missed mcast
    765152169    643629      0      0      0      0
   TX: bytes packets errors dropped carrier collsns
    80961641    476047      0      0      0      0
```

```
ip link set eth0 up
```

Activates the interface eth0

```
ip link set eth0 down
```

Deactivates the interface eth0

```
ip link set eth0 mtu 1400
```

Changes the MTU to 1400 bytes

```
ip link set eth0 name ens33
```

Renames eth0 to ens33

```
ip -brief link
```

Displays a clean, one-line summary of all interfaces

```
hosni ~ 16:16 ip -brief link
lo UNKNOWN 00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
eno1 DOWN f0:1f:af:59:ea:5a <NO-CARRIER,BROADCAST,MULTICAST,UP>
wlp3s0 UP 34:23:87:06:ee:69 <BROADCAST,MULTICAST,UP,LOWER_UP>
virbr0 DOWN 52:54:00:9c:06:84 <NO-CARRIER,BROADCAST,MULTICAST,UP>
ham0 UNKNOWN 7a:79:00:00:00:00 <BROADCAST,MULTICAST,UP,LOWER_UP>
vmnet1 UNKNOWN 00:50:56:c0:00:01 <BROADCAST,MULTICAST,UP,LOWER_UP>
vmnet2 UNKNOWN 00:50:56:c0:00:02 <BROADCAST,MULTICAST,UP,LOWER_UP>
vmnet3 UNKNOWN 00:50:56:c0:00:03 <BROADCAST,MULTICAST,UP,LOWER_UP>
vmnet4 UNKNOWN 00:50:56:c0:00:04 <BROADCAST,MULTICAST,UP,LOWER_UP>
vmnet5 UNKNOWN 00:50:56:c0:00:05 <BROADCAST,MULTICAST,UP,LOWER_UP>
vmnet6 UNKNOWN 00:50:56:c0:00:06 <BROADCAST,MULTICAST,UP,LOWER_UP>
vmnet7 UNKNOWN 00:50:56:c0:00:07 <BROADCAST,MULTICAST,UP,LOWER_UP>
vmnet8 UNKNOWN 00:50:56:c0:00:08 <BROADCAST,MULTICAST,UP,LOWER_UP>
vmnet9 UNKNOWN 00:50:56:c0:00:09 <BROADCAST,MULTICAST,UP,LOWER_UP>
vmnet10 UNKNOWN 00:50:56:c0:00:0a <BROADCAST,MULTICAST,UP,LOWER_UP>
vmnet11 UNKNOWN 00:50:56:c0:00:0b <BROADCAST,MULTICAST,UP,LOWER_UP>
vmnet12 UNKNOWN 00:50:56:c0:00:0c <BROADCAST,MULTICAST,UP,LOWER_UP>
```

USEFUL OPTIONS

Option	Description
<code>-h</code>	Show help for this object
<code>-s</code>	Display detailed statistics
<code>-d</code>	Show extended details (queueing, qdisc, etc.)
<code>-brief</code>	Compact one-line output
<code>-c</code>	Colorize output for better readability

OBJECT: ADDRESS

After mastering `ip link`, it's time to move one layer up — IP addresses. Every interface needs one (or more) to communicate, and the `ip address` object — often shortened to `ip addr` — is your key to view, add, or remove them.

Forget `ifconfig`; this is how Linux handles addresses today.

WHAT IT DOES

The `address` (or `addr`) object manages the assignment of IP addresses to network interfaces.

It lets you:

- View existing IPv4/IPv6 addresses
- Assign new addresses to interfaces
- Remove addresses
- Display subnet and broadcast information

Essentially, it controls where your system lives on the network.

BASIC SYNTAX

```
ip address [ show | add | del | flush ] [dev DEVICE] [options]
```

Or simply:

```
ip addr ...
```

SIMPLE EXAMPLE

Show all IP addresses on the system:

```
ip addr show
```

```
hosni ~ 19:33 ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eno1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether f0:1f:af:59:ea:5a brd ff:ff:ff:ff:ff:ff
    altname enp0s25
3: wlp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 34:23:87:06:ee:69 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.149/24 brd 192.168.0.255 scope global dynamic noprefixroute wlp3s0
        valid_lft 84332sec preferred_lft 84332sec
    inet6 fe80::c0de:57f9:13d:a77b/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
4: virbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default qlen 1000
    link/ether 52:54:00:9c:06:84 brd ff:ff:ff:ff:ff:ff
```

Assign a new IP address to an interface:

```
ip addr add 192.168.1.10/24 dev eth0
```

Remove an IP address:

```
ip addr del 192.168.1.10/24 dev eth0
```

COMMON COMMANDS EXPLAINED

Command	Description
<pre>ip addr show</pre>	Lists all addresses and their assigned interfaces

<code>ip addr show dev eth0</code>	Shows addresses for a specific interface
<code>ip addr add 192.168.1.10/24 dev eth0</code>	Adds an IPv4 address to eth0
<code>ip addr add 2001:db8::1/64 dev eth0</code>	Adds an IPv6 address
<code>ip addr del 192.168.1.10/24 dev eth0</code>	Removes a specific IP
<code>ip addr flush dev eth0</code>	Removes all addresses from eth0
<code>ip -brief addr</code>	Displays a compact summary of interfaces and addresses

USEFUL OPTIONS

Option	Description
<code>-h</code>	Show help for the address object
<code>-s</code>	Show statistics for interfaces
<code>-brief</code>	Compact and readable output
<code>-4</code>	Show only IPv4 addresses
<code>-6</code>	Show only IPv6 addresses
<code>-c</code>	Colorize output for easier reading

OBJECT: ROUTE

Once you've set up interfaces and assigned IP addresses, it's time to define where your packets go.

That's the job of the ip route object — it manages the kernel's routing table and decides the next hop for each destination.

It's one of the most powerful and critical tools in networking.

WHAT IT DOES

The route object handles routing tables — the rules your system follows to reach other networks.

It allows you to:

- View the current routing table
- Add or delete static routes
- Configure default gateways
- Troubleshoot connectivity issues

Every time you send data to another network, ip route is silently making the decision on how it gets there.

BASIC SYNTAX

```
ip route [ show | add | del | flush ] [OPTIONS]
```

SIMPLE EXAMPLE

View your current routing table:

```
ip route show
```

```
hosni ~ 19:33 ip route show
default via 192.168.0.1 dev wlp3s0 proto dhcp metric 600
169.254.0.0/16 dev virbr0 scope link metric 1000 linkdown
172.16.0.0/24 dev vmnet7 proto kernel scope link src 172.16.0.1
172.16.1.0/24 dev vmnet3 proto kernel scope link src 172.16.1.1
172.16.2.0/24 dev vmnet4 proto kernel scope link src 172.16.2.1
172.16.3.0/24 dev vmnet5 proto kernel scope link src 172.16.3.1
172.16.4.0/24 dev vmnet6 proto kernel scope link src 172.16.4.1
172.16.6.0/24 dev vmnet9 proto kernel scope link src 172.16.6.1
172.16.79.0/24 dev vmnet11 proto kernel scope link src 172.16.79.1
192.168.0.0/24 dev wlp3s0 proto kernel scope link src 192.168.0.149 metric 600
192.168.4.0/24 dev vmnet2 proto kernel scope link src 192.168.4.1
192.168.12.0/24 dev vmnet1 proto kernel scope link src 192.168.12.1
```

Add a new route to a specific network:

```
ip route add 10.0.0.0/24 via 192.168.1.1
```

Delete that route:

```
ip route del 10.0.0.0/24 via 192.168.1.1
```

COMMON COMMANDS EXPLAINED

Command	Description
<pre>ip route show</pre>	Displays all routing entries in the kernel
<pre>ip route show default</pre>	Shows the default route(s)
<pre>ip route add 10.0.0.0/24 via 192.168.1.1</pre>	Adds a route to reach the 10.0.0.0 network

<code>ip route add default via 192.168.1.1</code>	Sets the default gateway
<code>ip route del 10.0.0.0/24 via 192.168.1.1</code>	Deletes a specific route
<code>ip route flush table main</code>	Removes all routes from the main routing table

USEFUL OPTIONS

Option	Description
<code>-h</code>	Show help for the route object
<code>-4 / -6</code>	Show IPv4 or IPv6 routes only
<code>show table all</code>	Display all routing tables
<code>show cache</code>	Display route cache entries
<code>-c</code>	Colorize output for easier reading

OBJECT: NEIGH

After exploring interfaces, addresses, and routes, it's time to see how Linux actually finds other machines on the network.

That's where the `ip neigh` object comes in — it manages the neighbor (ARP/ND) table, which maps IP addresses to MAC addresses.

This is how your system remembers “who's who” on the local network.

WHAT IT DOES

The `neigh` (short for neighbor) object displays and manipulates the ARP table for IPv4 and the neighbor discovery cache for IPv6.

It allows you to:

- View all cached IP-to-MAC address mappings
- Add or delete static ARP entries
- Flush outdated entries
- Diagnose connectivity issues caused by stale ARP data

Whenever your system communicates on a LAN, it uses this table to resolve IPs into hardware (MAC) addresses.

BASIC SYNTAX

```
ip neigh [ show | add | del | replace | flush ] [ dev DEVICE ]  
[ options ]
```

SIMPLE EXAMPLE

Show all current neighbor (ARP) entries:

```
ip neigh show
```

```
hosni ~ 21:44 ip neigh show  
192.168.0.1 dev wlp3s0 lladdr fc:3f:fc:31:b8:12 REACHABLE
```

Each state (REACHABLE, STALE, DELAY, FAILED) tells you how recently the system confirmed communication with that host.

Add a static ARP entry:

```
sudo ip neigh add 192.168.1.50 lladdr 08:00:27:11:22:33 dev  
eth0 nud permanent
```

COMMON COMMANDS EXPLAINED

Command	Description
<pre>ip neigh show</pre>	Lists all current ARP/neighbor cache entries
<pre>ip neigh show dev eth0</pre>	Displays neighbor table for a specific interface
<pre>ip neigh add 192.168.1.50 lladdr 08:00:27:11:22:33 dev eth0 nud permanent</pre>	Adds a static ARP entry (won't expire)

<code>ip neigh del 192.168.1.50 dev eth0</code>	Deletes a specific neighbor entry
<code>ip neigh flush all</code>	Clears the entire neighbor table
<code>ip -brief neigh</code>	Displays a compact summary (available on newer systems)

USEFUL OPTIONS

Option	Description
<code>-h</code>	Show help for the neigh object
<code>-4 / -6</code>	Show IPv4 or IPv6 neighbor entries
<code>nud reachable/stale/permanent</code>	Set the Neighbor Unreachability Detection state
<code>flush</code>	Remove dynamic or all entries
<code>-c</code>	Colorize output for better readability

OBJECT: RULE

we've seen how to configure interfaces, assign addresses, manage routes, and check neighbors.

But what if you want to tell Linux which routing table to use based on conditions — like source IP, destination, or interface?

That's where the ip rule object shines.

It's the key to policy-based routing, giving you fine-grained control over how traffic is routed.

WHAT IT DOES

The rule object defines the routing policy database (RPDB) — a list of rules that determine which routing table should be used to look up routes.

While the default system uses the main table, ip rule allows you to:

- Route traffic differently based on source IPs, interfaces, or marks
- Use multiple routing tables
- Build advanced network topologies (multi-homing, VPNs, isolated routing domains)

BASIC SYNTAX

```
ip rule [ add | del | flush | show ] [ selectors ] [ table  
TABLE_ID ]
```

SIMPLE EXAMPLE

View all routing rules:

```
ip rule show
```

```
hosni ~ 21:53 ip rule show
0:      from all lookup local
32766:  from all lookup main
32767:  from all lookup default
```

Add a rule to route traffic from a specific source IP using a custom table:

```
sudo ip rule add from 192.168.1.50/32 table 100
```

Delete that rule:

```
sudo ip rule del from 192.168.1.50/32 table 100
```

COMMON COMMANDS EXPLAINED

Command	Description
<pre>ip rule show</pre>	Lists all routing policy rules in order of priority
<pre>ip rule add from 192.168.1.50/32 table 100</pre>	Routes traffic from this IP through table 100
<pre>ip rule add to 10.0.0.0/24 table 200</pre>	Routes traffic destined for 10.0.0.0/24 via table 200

<code>ip rule add fwmark 10 table 300</code>	Uses firewall mark (from iptables or nftables) to select a table
<code>ip rule del from 192.168.1.50 table 100</code>	Deletes a specific rule
<code>ip rule flush</code>	Clears all custom routing rules (use with caution)

USEFUL OPTIONS

Option	Description
<code>-h</code>	Show help for the rule object
<code>from</code>	Match packets by source address
<code>to</code>	Match packets by destination address
<code>iif / oif</code>	Match packets by incoming or outgoing interface
<code>fwmark</code>	Match packets by firewall mark
<code>table</code>	Specify which routing table to use
<code>priority</code>	Define rule precedence (lower = higher priority)



You can create multiple routing tables by adding entries in:
`/etc/iproute2/rt_tables`

Example:

```
100 internal
200 vpn
```

Then, associate them with rules:

```
ip rule add from 192.168.1.0/24 table internal
ip rule add from 10.8.0.0/24 table vpn
```

This setup lets you route corporate traffic through VPNs while keeping other traffic direct — all automatically.